



The Computer Should Feel Closer

A language designed to create momentum, not consume it.

Most people have more ideas than they have tools to express them. Programming can close that gap - but only when the language does not demand that people translate their thinking into ceremony before they can act.

Basic Next is designed around a simple belief: **a programming language should create momentum, not consume it.**

Flow is a product requirement

When a programmer is in flow, the next step is understandable, the feedback is nearby, and attention stays with the problem. BN is designed to protect that state.

- Every block closes in plain sight: `END IF`, `END WHILE`, `END FUNCTION`.
- Every variable introduces its type locally: `LET total AS FLOAT = 0.0`.
- One concept has one normal spelling.
- The source reflects meaningful execution steps instead of hiding them behind magic.
- The official tool has one simple path: check, run, build - with the same diagnostic language at every stage.

Clarity is not a compromise

Clear code can still have types, contracts, objects, modules, memory boundaries, and host capabilities. The point is not to hide complexity that matters. The point is to make complexity arrive only when it is needed.

A learning curve, not a cliff

BN uses a visible complexity ladder. A learner can start with typed values, conditions, loops, functions, and console input. Classes, interfaces, modules,

host capabilities, typed memory, data facilities, and parallel devices come later - as tools for real needs, not obstacles at the door.

That makes BN relevant in the classroom and beyond it. The same language that helps someone understand a loop can help them write a practical utility, express a domain model, or build a system that talks to its environment.

The standard

BN should feel calm, capable, and direct. It should make a reader think about the problem, not decode the language. That is the standard against which every feature should be judged.